

Cartman PH — Mobile Applications

Product Backlog

Tech Lead Note: This backlog covers the mobile development scope only (Customer App). All web applications (Admin Dashboard, Merchant Panel, Financial Ledger) are excluded from this tracking and will be handled separately.

1. Customer Ordering App (Core Scope)

Target Stack: Flutter or React Native, geolocator/react-native-geolocation-service, flutter_map/react-native-maps (OpenStreetMap Tile Servers), Local Storage (Hive or AsyncStorage), Supabase Auth, and Firebase Cloud Messaging (FCM).

Epic: Authentication & Onboarding

Task C-1.1: Supabase Auth Integration & Registration

- **User Story:** As a Customer, I want to sign up and log in using my mobile phone number or email, so that I can securely manage my profile, checkout details, and order history.
- **Technical Context:** Use **Supabase Authentication & GoTrue** to manage identity records and issue secure JSON Web Tokens (JWT) for API validation.
- **Acceptance Criteria:**
 - User can input their phone number/email to trigger a registration request.
 - Session tokens must be securely stored on the device for persistent logins.

Task C-1.2: Transactional SMS Verification (OTP)

- **User Story:** As a Customer, I want to receive a one-time PIN (OTP) via SMS when signing up, so that my mobile number is verified before I place cash orders.
- **Technical Context:** Write a **Supabase Edge Function** (Deno runtime) that triggers upon user registration to interface with the third-party **Semaphore SMS API** gateway.
- **Acceptance Criteria:**
 - An SMS containing a temporary verification code is successfully sent to the user's phone.
 - The mobile app blocks further progress until the correct OTP is entered.

Epic: Geolocation & Mapping Infrastructure

Task C-2.1: Hardware GPS Triangulation

- **User Story:** As a Customer, I want the app to automatically detect my current physical location when opened, so that I don't have to manually type my provincial address.

- **Technical Context:** Integrate the native hardware GPS layer using **geolocator** (for Flutter) or **react-native-geolocation-service** (for React Native).
- **Acceptance Criteria:**
 - App requests location permissions transparently on first launch.
 - Accurately grabs device geocoded coordinates with minimal latency.

Task C-2.2: Open-Source Map Layout Rendering

- **User Story:** As a Customer, I want to see my drop-off point displayed on an interactive map and adjust it by dragging a pin, so that the rider gets exact coordinates.
- **Technical Context:** Initialize **flutter_map** or **react-native-maps** bound directly to open-source **OpenStreetMap raster arrays / Tile Servers**. Do *not* use expensive Mapbox/Google Maps APIs for Phase 1 to preserve the ₱0 infrastructure cost baseline.
- **Acceptance Criteria:**
 - Map loads efficiently without relying on enterprise tier API keys.
 - Moving the pin extracts the final latitude and longitude coordinates to pass to the checkout payload.

Epic: Cart & Checkout Core

Task C-3.1: Offline-First Local Cart Caching

- **User Story:** As a Customer, I want my selected items to remain intact in my cart even if my data connection drops while browsing provincial areas, so that I don't lose my selections.
- **Technical Context:** Utilize **Hive** (Flutter) or **AsyncStorage** (React Native) to handle client-side order transaction caching.
- **Acceptance Criteria:**
 - Adding, removing, or modifying items mutates local device storage.
 - Closing or backgrounding the app does not wipe the current cart state.

Task C-3.2: Relational Order Placement (COD Focus)

- **User Story:** As a Customer, I want to select "Cash On Delivery" (COD) and submit my order, so that the system logs my transaction and notifies nearby operations.
- **Technical Context:** Upon checkout, send the payload to the centralized **orders** table in **Supabase PostgreSQL 15+**. The schema must capture initial states (**pending**), total costs, geocoded coordinates, and proper foreign keys tied to the user metadata.
- **Acceptance Criteria:**
 - Successfully writes a new record to the **orders** table.
 - Enforces strict foreign key constraints to prevent orphan data.

Epic: Order Lifecycle & Realtime Telemetry

Task C-4.1: WebSockets Order Status Tracking

- **User Story:** As a Customer, I want to see my order status update instantly on my screen (e.g., from *Preparing* to *Picked Up*), so that I know exactly what stage my delivery is in.
- **Technical Context:** Connect the mobile client via **@supabase/supabase-js** to listen directly to database WebSockets via **Supabase Realtime**. This listens to the PostgreSQL Write-Ahead Log (WAL) to avoid heavy HTTP polling.
- **Acceptance Criteria:**
 - UI dynamically updates the state banner immediately when a merchant or rider changes the order status row in the database.

Task C-4.2: Firebase Cloud Messaging (FCM) Alerting

- **User Story:** As a Customer, I want to receive a push notification when my food is arriving even if my phone is locked, so that I don't miss the rider.
- **Technical Context:** Integrate the **FCM wrapper libraries** within the mobile client to handle background native device alerting.
- **Acceptance Criteria:**
 - Receives system alerts flawlessly when the application scope is minimized or dead.

Epic: Core Ordering & Requests

Task C-5.1: Food Ordering Module

- **User Story:** As a Customer, I want to browse a merchant's menu by categories, select variations/add-ons, and add items to my cart, so that I can customize my food orders digitally.
- **Technical Context:** Fetch menu items dynamically from the **merchants** and menu buckets in Supabase Storage. Manage item states and custom modifiers (e.g., extra sauce, size options) using local state management before cart submission.
- **Acceptance Criteria:**
 - Users can view categorized items with active stock states.
 - Adding items with variations updates the running cart total accurately.

Task C-5.2: Pickup & Delivery Requests (Errands)

- **User Story:** As a Customer, I want to create a custom "Errand" or "Pabili" request by typing out specific items and store instructions, so that I can get non-food items delivered.
- **Technical Context:** Create an alternative order type flow within the **orders** table. This payload will write custom text descriptions into a JSONB column rather than linking structured merchant menu IDs.
- **Acceptance Criteria:**

- Form fields capture: Store Name, Item Description List, and Estimated Budget.
- Successfully maps to an "Errand" category in the backend dispatch system.

Task C-5.3: On-Demand Rider Booking

- **User Story:** As a Customer, I want to book a rider to deliver a package from my current location to another local address, so that I can use the platform as a pure courier service.
- **Technical Context:** Require both origin and destination geocoded coordinates using the open-source map engine. Calculate distance client-side or via a Supabase Edge Function to generate the base delivery fee.
- **Acceptance Criteria:**
 - App captures a distinct Pickup Address and Drop-off Address.
 - Calculates and displays the localized delivery fee before the user taps confirm.

Epic: Checkout Details & Communication

Task C-6.1: Order Notes & Custom Instructions

- **User Story:** As a Customer, I want to add specific notes to my order (e.g., "Make it spicy," or "Leave at the blue gate"), so that the merchant and rider understand special requirements.
- **Technical Context:** Map text input fields to `merchant_notes` and `rider_notes` columns in the `orders` schema.
- **Acceptance Criteria:**
 - Character-limited text area on checkout.
 - Data is saved reliably and streams to the Merchant Web Panel and Rider App via WebSockets.

Task C-6.2: Delivery Address Input & Saved Locations

- **User Story:** As a Customer, I want to type in my text address details (e.g., Barangay, landmarks, house number) along with my map pin, so that the rider has local context.
- **Technical Context:** Relate a text-based address profile table to the user's Supabase Auth identity metadata.
- **Acceptance Criteria:**
 - Validates that text address info is completed before checkout is unlocked.
 - Allows saving locations as "Home", "Work", or "Current".

Task C-6.3: Contact Information Binding

- **User Story:** As a Customer, I want my verified mobile phone number and recipient name attached to the order, so that the rider can contact me upon arrival.
- **Technical Context:** Automatically inject the verified phone number from the Supabase Auth profile metadata directly into the active order payload.
- **Acceptance Criteria:**

- Rider can view a "Call Customer" click-to-dial event using native device phone dialer linking.

Epic: Order History & Status Tracking

Task C-7.1: Granular Step-by-Step Tracking

- **User Story:** As a Customer, I want to see exactly which phase my order is in (*Pending, Preparing, Rider Arrived at Merchant, In Transit, Delivered*), so that I am kept informed without manual communication.
- **Technical Context:** Build a visual stepper UI that listens to the **orders** status column updates via Supabase Realtime.
- **Acceptance Criteria:**
 - The stepper advances seamlessly in real-time as the order state transitions.

Task C-7.2: Basic Customer Order History

- **User Story:** As a Customer, I want to view a list of my past completed and canceled orders, so that I can quickly re-verify what I spent or re-order past items.
- **Technical Context:** Query the **orders** table filtered by the current user's authenticated JWT ID, ordered by timestamp descending. Limit to the latest 20 items for initial optimization.
- **Acceptance Criteria:**
 - Displays summary card containing: Merchant Name, Date, Total Paid, Status, and Item Summary.